EXPERIMENT NO. 3

Sum of Squares of Distinct Counts of Subarrays

Programming Language: Java

Name : A Yuvan Charan
Register No. : 192424383

Aim

To write a Java program that returns the sum of the squares of the distinct counts of all subarrays of a given array.

Question

You are given a 0-indexed integer array nums. The distinct count of a subarray nums[i..j] (0 <= i <= j < nums.length) is the number of distinct values in it. Return the sum of the squares of distinct counts of all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1

Input: nums = [1,2,1]

Output: 15

Explanation: [1], [2], [1] have 1 distinct value each; [1,2], [2,1], [1,2,1] have 2 distinct values each. The sum is 1^2 + 1^2 + 1^2 + 2^2 + 2^2 + 2^2 = 15.

Example 2

Input: nums = [1,1]

Output: 3

Explanation: [1], [1] and [1,1] each have 1 distinct value, so the sum is 1^2 + 1^2 + 1^2 = 3.

Algorithm

Step 1: Start.

Step 2: Read the array nums of size n and set total = 0.

Step 3: For each starting index i from 0 to n-1, clear a marker array seen[] and set distinct = 0.

Step 4: For each ending index j from i to n-1, if nums[j] has not been seen, mark it and increment distinct.

Step 5: Add distinct * distinct to total (this is the square of the distinct count of nums[i..j]).

Step 6: After all subarrays are processed, print total.

Step 7: Stop.

Program (Java)

```
public class Main {

    // Assumes 1 <= nums[i] <= 100
```

```
    static int sumOfSquares(int[] nums) {
        int n = nums.length, total = 0;
        for (int i = 0; i < n; i++) {
            boolean[] seen = new boolean[101];
            int distinct = 0;
            for (int j = i; j < n; j++) {
                if (!seen[nums[j]]) {
                    seen[nums[j]] = true;
                    distinct++;
                }
                total += distinct * distinct;
            }
        }
        return total;
    }

    public static void main(String[] args) {
        int[] nums1 = {1, 2, 1};
        int[] nums2 = {1, 1};

        System.out.println("Output: " + sumOfSquares(nums1));   // Example 1
        System.out.println("Output: " + sumOfSquares(nums2));   // Example 2
    }
}
```

Output

```
Output: 15
Output: 3

=== Code Execution Successful ===
```

Complexity Analysis

Time Complexity: O(n^2) - every subarray is visited once and the distinct count is updated in O(1).
Space Complexity: O(1) - the marker array has a fixed size of 101.

Result

Thus the program was executed successfully and the output was verified.